

Research Paper Synthesis Graph

1 What is this problem?

Before contributing to a field, a researcher must establish what has been tried, what is still disputed, and what nobody has attempted yet — work that now means reading several hundred papers and is repeated independently by every newcomer. The bottleneck is not access to papers but *synthesis* across them, and the most valuable relationships are precisely the ones no single paper states: one paper introduces a method, another reports where it fails, and the connection exists in the literature as a whole without existing in any document within it. The same gap makes disagreement hard to see, since a paper rarely contradicts itself, and makes gap-finding harder still, because a question nobody has attempted looks identical in the text to one that was attempted and abandoned. The motivation for this project is a concrete instance of that difficulty: my master’s thesis, *Using Error-Correcting Codes to Encode Colors in the Graph Community Detection Problem*, sits across quantum computing and graph community detection and demanded exactly this kind of cross-literature reading. That topic accordingly forms one part of the evaluation set — 10 of the 34 reference questions — with the remainder drawn from quantum computing research more broadly. The domain suits the problem well: it moves quickly, its central claims are actively contested, its results depend on hardware that differs between laboratories, and much of it concerns what is not yet possible.

2 Why Graph-based RAG (structured retrieval)?

Because the research problem is *relational*. A lexical or vector RAG system answers “*what is this paper about*” well: the answer sits in a single passage, and similarity search finds it. But “*what methods have been tried on problem X, and which of them were limited by Y?*” has an answer that lives in the relationships *between* papers, methods and limitations. No single passage states it, so similarity search can at best assemble it by luck.

The standard response is to make those relationships explicit: extract a typed graph in which *method M addresses problem P* and *finding F undercuts M* are edges, so that a multi-hop question becomes a traversal rather than a similarity guess [1]. **Whether that response actually works is the question this work measures, not an assumption it rests on.**

3 Iteration 1

3.1 Design/Coding Process

1. Create directory structure + project meta files (pyproject, README, gitignore, env, Makefile, configs)
2. Core library: config, logging, extraction schema (frozen tiered schema)
3. Store interfaces + Kuzu/vector adapters (Phase-2 portable)
4. *Ingestion*: S2 client, ArXiv, PDF parser, section aware chunking
5. *Extraction*: prompts + API-based extractor

6. *Evaluation harness* — gold schema, metrics, judge, calibration, runner + baselines.
7. Pipeline scripts, sample gold data, and tests.

3.2 Pipeline (sketch)

1. **Ingest** — fetch Tier-A metadata (title, authors, venue, references) from the Semantic Scholar Graph API, download PDFs from arXiv, parse them into typed sections, and split each section into retrieval-sized chunks.
2. **Extract** — run a section-routed LLM extractor (`claude-haiku-4-5`, structured JSON output) over every (paper, section) to emit typed nodes (`Method`, `Problem`, `Claim`, `Limitation`, `Dataset`, `Hardware`, ...) and typed edges (`ADDRESSES`, `BUILDS_ON`, `REFUTES`, `UNDERCUTS`, `EVALUATED_ON`, ...), each carrying an `evidence_quote`, a `confidence`, and paper provenance.
3. **Build graph** — normalize surface names into stable node ids (same normalized string $\Rightarrow$ one node), stitch edges by resolved endpoints, and index chunk embeddings in the vector store; metadata nodes come from Semantic Scholar, never from the extractor.
4. **Retrieve** — for a query, either run vector RAG over abstracts / full-text (the baselines) or traverse the typed synthesis graph to gather the relevant subgraph and its supporting evidence.
5. **Synthesize** — feed the retrieved evidence to the synthesis LLM for a query-focused, per-claim-cited answer that hedges on thin evidence and surfaces contradictions.
6. **Evaluate** — score every system over a structured gold set with deterministic metrics (citation recall/precision, refutation surfacing) plus a calibrated LLM judge, reporting results *by query type* (relational and refutation queries are where the typed graph earns its complexity).

3.3 Terms

1. **GROBID** — *GeneRation Of BIbliographic Data*. An open-source machine-learning tool that extracts and structures information from scholarly documents, primarily PDFs.
2. **Cypher** — a query language for graph databases (e.g. Neo4j).
3. **Kuzu** — an embedded graph database. Runs in-process, stores the graph in a local directory, and speaks Cypher — much like SQLite does for relational data.
4. **KuzuGraphStore** — the adapter that lets a RAG pipeline read from and write to a Kuzu graph. This talks about "what cites what".

(a) Setup: The whole store is one node table (`Entity`) and one edge table (`REL`). Everything a Paper, an Author, a "cites" link - is a row in one of these tables.

(b) The mental model:

```

init_schema()  →  make the 2 tables (once)
    ↓
upsert_nodes() →  MERGE rows into Entity (nodes first!)
    ↓
upsert_edges() →  MATCH endpoints, MERGE into REL

```

↓
 query() → Cypher in, list[dict] out
 ↓
 staged (written by the agent at query time, untrusted) →
 curated (reviewed, what metrics run against)
 promote_staged() → - explicit trust boundary

5. **FAISS** — The problem of finding the most similar vector when there comes any query in a form of vector. FAISS is C++ based and does the O(n) similarity search faster, also allows ANN based search.

6. **FaissVectorStore** — Talks about "what text is this about".

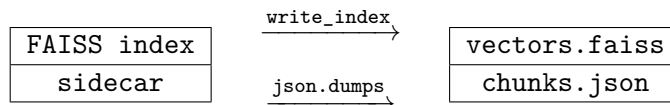
- (a) The RAG pipeline uses vector search to find candidate passages, then the graph store to expand and verify relationships around them.
- (b) FAISS is a B-tree, stores/indexes the vectors. FAISS only deals with numbers, to lookup the numbers/indices, we keep a python list or a sidecar which stores all the chunks which are useful.

query vector
 ↓
 FAISS index "closest rows: 7, 2, 41, 3, 19"
 ↓ *just integers*
 sidecar [7]->gpt3#c4 [2]->bert#c1 [41]->ppo#c2 ...
 ↓ *now they're Chunks — with corpus, paper_id, section_type*
 filter corpus=="fulltext", truncate to top_k
 ↓
 list[SearchHit]

- (c) Persistence

in memory

on disk



must be written and read TOGETHER

- (d) Mental Model —

add()	append to BOTH, same order	→ invariant preserved
search()	FAISS gives ints → sidecar gives Chunks	→ skip -1, filter corpus
persist()	two files, written together	→ temp+rename to survive crashes
load()	two files, read together	→ assert alignment
delete	don't, or use IndexIDMap2	→ compaction destroys positions

7. **Embedding matrix vs Vector DS** — Embedding matrix contains random vectors, and is a learned lookup table of token vectors inside the model; and Vector Store contain similar

vectors depending on the context. E.g., "bank" in river bank and investment bank gets the same row in Embedding matrix but produce different stored vectors in DB.

8. **Embedder** — Chunks are embedded with SPECTER2 (allenai/specter2_base, 768-d), a sentence-transformer trained on scientific papers using citation signal, so retrieval matches on topical relatedness rather than surface wording. A dependency-free HashEmbedder is available as a deterministic stand-in for offline smoke tests, but is explicitly excluded from reported results.
9. **Section-aware chunking** — Papers are split into retrievable chunks by packing whole sentences greedily up to a 512-token budget with a 64-token overlap, never crossing a section boundary — so each chunk keeps a single section label, which acts as a strong prior on what kind of fact it contains (limitations in Discussion, hardware details in the Appendix). Token counts are estimated as words / 0.75, a rule of thumb borrowed from everyday English that likely understates token counts on dense technical prose; since the SPECTER2 embedder truncates hard at 512 tokens, chunks may be silently cut short. Verifying this needs only a ten-line comparison against SPECTER2's own tokeniser, making it a concise and self-contained addition to the write-up (Future work).
10. **Schema** — Node and edge types are stratified by extraction reliability: Tier A metadata (Paper, Author, cites) arrives free from the Semantic Scholar API and is near-exact, Tier B semantic nodes (Method, Problem, Limitation) are LLM-labelled at medium precision, and Tier C relational edges (addresses, limited_by, refutes) are LLM-inferred and least reliable — yet carry most of the system's value, since Tier A alone reduces to a citation network. The governing rule is that the hard tier must degrade rather than block: where Tier C edges are absent, the relational traversal falls back to same-paper co-occurrence and explicitly labels the weaker provenance so the synthesiser hedges accordingly.
11. **Semantic Scholar Graph API client** — Fetches paper metadata and citation links from the Semantic Scholar Graph API, the pipeline's primary (Tier-A) metadata source.
12. **ArXiv PDF retrieval** — Downloads paper PDFs from arXiv so their full text can be extracted into sections and chunked.
13. **PDF Parser** — It takes the PDF that `arxiv_client` downloaded, extracts the raw text, and splits it into structured sections — Abstract, Introduction, Methods, Results, etc. — as `Section` objects, which `chunking.py` then breaks into chunks for embedding. It's the bridge from a binary PDF to clean, section-labeled text.
14. **Prompts** — The LLM extraction prompts — a shared system prompt plus a per-section-type user prompt that tells the model exactly which node/edge types to pull from that section (e.g., limitations from Discussion, hardware/reproducibility facts from Appendix), so extraction stays focused and precise.
15. **Extractor** — Extracts typed graph nodes and edges from each paper section via the Claude API (structured JSON output), then normalizes names into stable node ids and stitches edges to build the paper's subgraph.
16. **Gold Schema** — It defines the data model for the evaluation gold set, the hand-written "correct answers" to be scored against the system. Each `GoldRecord` is one benchmark query with structured skeleton so answers can be graded both automatically and by an LLM judge.
17. **Metrics** — Computes the deterministic; LLM-free evaluation scores (must-cite recall, citation

precision, key-claim source recall, and refutation-surfacing) that can be verified exactly. Note: The formulae updated in iteration 2 incorporating the usecase when there’s no corresponding text field/empty value to field in gold schema, also the usecase when there’s no answer produced via model.

Let C = the set of paper ids the answer cited, $M = \text{gold.must_cite}$, $K = \text{gold.key_claims}$, $R = \text{gold.known_refutations}$.

must_cite_recall

$$\text{must_cite_recall} = \begin{cases} \text{None} & M = \emptyset \\ \frac{|M \cap C|}{|M|} & \text{otherwise} \end{cases}$$

citation_precision

Let $\mathbf{Rel} = M \cup \bigcup_{k \in K} k.\text{papers}$ — the union of required papers and every key-claim source.

$$\text{citation_precision} = \begin{cases} 1.0 & C = \emptyset \\ \text{None} & C \neq \emptyset, \mathbf{Rel} = \emptyset \\ \frac{|C \cap \mathbf{Rel}|}{|C|} & \text{otherwise} \end{cases}$$

The two guards are deliberately different. $C = \emptyset$ is a property of the answer — it cited nothing, so it is vacuously precise, and **no_citations** fires as a separate well-formedness violation. $\mathbf{Rel} = \emptyset$ is a property of the gold — nothing to be precise about.

key_claim_source_recall

$$\text{key_claim_source_recall} = \begin{cases} \text{None} & K = \emptyset \\ \frac{1}{|K|} \sum_{k \in K} 1[k.\text{papers} = \emptyset \vee k.\text{papers} \cap C \neq \emptyset] & \text{otherwise} \end{cases}$$

A claim with no source papers counts as a hit. That’s the free-credit case I flagged earlier and why **open-t02** was deleted — an unsourced **key_claim** scores correct against any answer.

refutation_surfaced

For a pair $r \in R$, let A_r and B_r be the **paper**:-prefixed tokens parsed out of **r.a** and **r.b** by **_papers_in**.

$$\text{refutation_surfaced} = \begin{cases} \text{None} & R = \emptyset \\ \frac{1}{|R|} \sum_{r \in R} 1[A_r \cap C \neq \emptyset \wedge B_r \cap C \neq \emptyset] & \text{otherwise} \end{cases}$$

Both sides must be cited. Citing one side scores 0 for that pair — surfacing half a contradiction isn’t surfacing it.

Aggregation

$$\overline{m} = \frac{1}{|\{q : m_q \neq \text{None}\}|} \sum_{q : m_q \neq \text{None}} m_q$$

`None` values are dropped, not coerced. The report prints the surviving count as `n` of `N` — that’s what turned `refutation_surfaced` from 0.800 over 10 queries into 0.333 over 3.

18. **Judge** — Runs an LLM-as-judge that grades each answer 1-5 on five qualitative criteria (coverage, attribution, hedging, accuracy, refutation handling) synthesis using a versioned prompt and structured output.
19. **Calibration** — Validates the judge’s trustworthiness against human ratings via quadratic-weighted Cohen’s kappa and Spearman correlation, and test for length bias, flagging any criterion that should not be trusted.
20. **Baselines** — Defines the System interface and the vector-RAG baseline systems(retrieve top-k chunks over abstracts or full-text, then LLM-synthesize a cited answer) that the typed-graph system is benchmarked against.
21. **Runner** — Executes a given system over the entire gold set and writes a scored timestamped run(answers, traces, deterministic + judge scores, and an overall + by-query-type `report.md`)

3.4 Running Tests

3.5 Key Design Decisions

3.5.1 Tier 1 — the differentiators

#	Design choice	Question it answers
1	Eval-first: scored baseline before building the system	“How do you know it works?” — shows you think like an ML engineer, not a demo-builder.
2	LLM-as-judge + calibration (quadratic-weighted κ , length-bias regression, self-preference guard)	“How do you evaluate generative output?” — very few candidates know judge bias exists; this is the standout.
3	Report broken down by query type (don’t let a mix average the effect away)	“How do you avoid fooling yourself?” — experimental/ablation rigor.
4	Tiered schema by extraction reliability (A/B/C, hard tier never blocks the system)	“What’s the actual bottleneck?” — shows you found the real hard problem (extraction is the quality ceiling).
5	API vs. local model for a one-time batch (\$20–45 buys out the bottleneck)	“Build vs. buy / local vs. API — how do you decide?” — cost/quality reasoning.
6	Curated vs. staged layers, no auto-merge	“How do you stop an agent from poisoning its own memory?” — data governance + failure-mode thinking.

3.5.2 Tier 2 — strong LLM/RAG + SWE fundamentals

#	Design choice	Question it answers
7	Structured outputs + schema-driven extraction	“How do you get reliable JSON out of an LLM?”
8	Dependency inversion / interfaces (Kuzu→Neo4j, FAISS→Qdrant is a config swap)	“How would you scale or migrate this?” — SOLID / ports-and-adapters.
9	Section-aware chunking (keep claim + evidence together; keep appendices)	RAG-specific competence beyond fixed-window chunking.
10	Deterministic metrics separated from the judge	“What can you defend without a model in the loop?”
11	Collapsing 7 agents → 3 roles + 4 tools (the architecture pushback)	“Tell me a time you resisted over-engineering” — a genuine design-tradeoff story.
12	Idempotent, resumable pipeline	Batch/data-engineering maturity.

4 Iteration 2

4.1 Design/Coding Process

Each step is a precondition for the next: traversal over unresolved ids walks into one of two nodes naming the same entity, and the reproducibility projection is empty until the **Hardware** layer exists.

1. **Resolve entities** — collapse surface-form duplicates (**MLP / Multilayer Perceptron (MLP)**) into a single id per entity, scoped within node type, emitted as an auditable id map rather than applied blind.
2. **Resolve query entry points** — match a query to a small set of typed nodes (**Problem: modularity maximization, Hardware: D-Wave**) instead of embedding it against the whole corpus. This is where dilution is avoided: the candidate set is chosen structurally before anything is ranked.
3. **Typed traversal** — expand from those entry nodes along typed edges (**ADDRESSES, BUILDS_ON, REQUIRES**), selected by query shape. Papers enter the candidate set because they are connected, not because they out-score 234 topically adjacent papers.
4. **Citation traversal** — expand along **CITES**. This doubles as the citation-graph baseline the typed arm is measured against.
5. **Reproducibility projection** — flatten **Hardware, Software** and **ReproducibilityArtifact** nodes into a per-paper field record, which is what **repro_gold** scores against.
6. **Score against Iteration 1** — same gold set, same judge. The bar is **must_cite_recall 0.367** at $k=60$, and the ranks the topic subset reached with the gold set in hand (2, 4, 7) reached here without it.

4.2 Terms

1. **Entity Resolution** — It looks for pairs that are obviously the same and writes down “these two should be one.” Two rules:
 - (a) **Normalization** — ignore cosmetic differences. Uppercase vs. lowercase, punctuation, a trailing bit in parentheses. `Transformer` and `transformer`. are the same thing.
 - (b) **Acronym expansion** — if the corpus only ever spells out an acronym one way, join the short form to the long form. Bare “VQE” gets folded into “Variational Quantum Eigensolver (VQE)”. But only if there’s exactly one expansion. 105 acronyms in this corpus are ambiguous — “PQC” appears with 11 different meanings — so those are left alone.
2. **Typed Retrieval** — The ordinary way to answer a research question is to search for passages that look similar to the question and write an answer from whatever comes back. That works when the answer sits in one passage.

It should work badly when it doesn’t. Ask “which methods tackle this problem, and what’s wrong with each” and no single passage holds both halves—you need one passage naming the methods and several others describing their weaknesses.

So we built a map instead. Pull the methods, problems, findings and limitations out of every paper. Record the connections the papers themselves state between them—“this method addresses that problem”, “this finding weakens that claim”. Then answer by walking the map: start at the problem, step to the methods, step again to their limitations. Assemble an answer no single passage contains.

How it works

Three steps:

- (a) Find a starting point. Compare the question against the short names of every item in the map, keep the closest twelve.
- (b) Walk outwards. Follow the links from those twelve, then follow the links again from wherever you land.
- (c) Write the answer from the supporting sentences attached to everything reached.

4.3 Fixes

In this, we are making the comparison meaningful before touching the graph, and improving the extraction quality.

1. **Evidence in traces** — Added code in script runners, evidence is being written into `traces.jsonl`
2. **Raising top_k and measure** — The number of output tokens returned as query is raised and script `run_eval` is executed we keep top_k similar chunks return on similarity checks. Settling up for top_k=60.
3. **Resolve corpus/gold mismatch** — We have noticed a few papers which are indexed and not being referred, and this is not an embedding problem. This is scoped target, we know the gold mismatch can be resolved narrowing down the papers its supposed to refer to, and won’t be ideal. In this iteration, *typed graph retrieval* is supposed to narrow by structure rather than by embedding similarity over everything. For example, if the question is: "How is the

modularity objective converted into a form solvable on quantum hardware, and what does each step of that conversion cost?", you do:

- (a) **Find Entry Points** – You don't embed that whole sentence against 10,993 prose chunks. You match it against node names, within type:- Problem: modularity maximization; Method: QUBO formulation; Hardware: D-Wave
 - (b) **Traversal** – Now the node Problem: modularity-maximization. Walk ADDRESSES backwards and every Method in the corpus that addresses it can be retrieved. Negre is in that set. Newman is in that set. They're in it because they're connected, not because they beat 141 quantum papers on cosine similarity. Rank stops being a corpus-wide competition.
And this is precisely why it survives the dilution: those 234 quantum papers are topically adjacent in embedding space, but the overwhelming majority have no ADDRESSES edge to modularity maximization. Traversal is a hard filter where similarity is a soft one. That's the whole difference.
 - (c) **Types from query shape** — The vector system flattens all of the query into one similarity score. The typed system reads it as an instruction about which edges to walk. rel-t09(Gold set) — "what quantum hardware did the D-Wave study use, and how many variables" — is almost literally a Paper → REQUIRES → Hardware lookup with a qubit_count field read off the node.
4. **Inapplicable metrics -> None** — Introduced None, while all the scores returning 1.0 when there's nothing returned from gold set evaluation or if the gold set field set is empty. It made metrics more honest.
 5. **Hardware routing** — Added NodeType.HARDWARE to results, method and availability.

5 Iteration 3

The system in this project answers research questions by searching a library of 271 papers and writing a cited answer. Iterations 1 and 2 built and tested a straightforward version: take the question, do one search, write the answer.

Iteration 2 found a specific weakness. The system did worst on what we call *relational* questions — questions with two linked parts, such as “*which methods have been tried on this problem, and what limits each one?*”. On those it scored 0.202 out of 1.0, against 0.357 for the simplest baseline. Iteration 2 also proposed a reason: the knowledge graph was missing the links that connect a method to its limitations, so the search could reach the first half of such a question but never the second.

That diagnosis left two possible repairs, and Iteration 3 tests both:

- **Add the missing links** to the graph, so the search can follow them.
- **Stop needing them:** split the question into two, search for each half separately, and let the second half be a second *search* rather than a step through the graph.

The short version of what happened: the second repair works, the first does not, and the reason the second works is not the reason we expected.

5.1 Design/Coding Process

The order the work was done in, and why

Iteration 3 had six pieces of work. The order was not chosen for convenience — each position protects against a specific way the final result could have come out meaningless.

1. **The agentic loop** — the thing being tested.
2. **Trajectory eval** — a way to measure the *plan* rather than the answer. Had to exist before the final experiment.
3. **Query-time write layer** — lets the system keep what it works out.
4. **Local inference** — running the AI model on our own hardware.
5. **The final experiment** — runs once, after the system is locked.
6. **The maintenance track** — four leftover problems from Iteration 2.

In practice the maintenance track was done *first*, not last, and that turned out to matter. It uncovered a fault in how the system’s answers were being marked (see Fixes, group 1). Had the new system been measured before that fault was found, the noise from the faulty marking would have been blamed on the new system.

1. The agentic loop

“Agentic” means the system decides what to do next rather than following one fixed path. The loop works like this:

1. **Plan.** Break the question into parts. “*Which methods, and what limits each?*” becomes a search for the methods and then a search for each method’s limitations.
2. **Search** once for each part.
3. **Check.** Compare what was found against the plan and name anything still missing.
4. **Search again** for whatever the check flagged.
5. **Write** the answer from everything gathered.

It plugs into the existing test harness without changing it, because that harness was deliberately written in Iteration 1 to accept any system that takes a question and returns an answer.

Three design decisions mattered more than the loop itself, because they are what make the eventual comparison honest.

It reuses the old system’s answer-writing code, rather than a copy of it. The new system holds an instance of the old one and calls its functions directly. This sounds fussy but it is the whole basis of the comparison: if the new system formatted its evidence or wrote its citations even slightly differently, a change in the score could be the *writing* rather than the *planning*. An automated test checks that both systems use literally the same function, so if someone later copies and edits it, the build fails.

The search budget refuses rather than warns. The system may make six searches per question. The seventh raises an error rather than logging a note. A limit that is only recorded is not a limit. Each search also returns 20 passages instead of the old system’s 60, so several searches together read roughly as much text as one old search — otherwise a better score could simply mean “it read more”.

When something breaks, it falls back *and says so*. If the planning step fails, the system does one plain search on the original question and sets a flag recording that it did. The flag matters more than the fallback: without it, a broken planner would quietly score as a working one.

2. Trajectory eval

Every measurement in the project so far scored the *answer*. None could tell apart a system that planned well and wrote badly from one that did the reverse — and for a system whose whole point is planning, that is exactly the distinction being studied. So this tool scores the plan.

It asks three things:

- Did the plan’s parts actually cover what the question asked?
- How many of the required papers did each search find? (A system can find more papers while spending far more searches, which is worse, not better.)
- Did the self-check step add a paper that was genuinely needed — or was it busy work?

Why it was built before the final experiment. Two reasons. The plan only exists while the system is running, so if it is not recorded at the time it cannot be recovered later — and the final experiment is only allowed to run once. And choosing which measurements to use *after* seeing the results is the same mistake as tuning a system on its own exam paper.

That ordering paid for itself twice, because building the tool exposed two faults that would otherwise have been inside the published result:

- One measurement returned zero for every single question, because two files wrote paper identifiers in slightly different formats and the comparison could never match. It looked like a real result, not an error.
- Another measurement was meaningless. It scored the plans a perfect 1.000. Testing it against *other questions’* plans revealed that unrelated plans passed just as easily — it was detecting that all phrases in this research field sound alike, not that the plans were good.

A deliberate choice: none of these measurements asks an AI model anything. They are all arithmetic. Elsewhere in this iteration we established how much work is needed to make an AI-based measurement trustworthy, so rather than validate a new family of AI-based measurements, we avoided needing to.

3. Query-time write layer

While answering, the system works something out — how to split the question, and which papers turned out to be relevant to each part. Previously all of that was thrown away the moment the answer was returned. This layer saves it.

Everything saved is marked *unreviewed* and is invisible to scoring. That separation is the entire point: if the system could add to the data it is scored against, it could improve its own marks. The

test for this feature is therefore not “does it work” but “*does it change nothing that is measured*” — an equality is a much stronger check of separation than any positive test.

That test found something. The rule “scoring only reads reviewed data” was written in the code from Iteration 1 onward, and **nothing had ever enforced it**. The promise held only because nothing had ever written unreviewed data. The first time the system wrote anything, it would have become false.

4. Local inference

The idea: run the language model on our own hardware instead of paying a provider, because an agentic system makes several model calls per question instead of one, so costs would rise.

Measurement showed the reasoning was wrong. The system makes three calls per question, not the five to ten assumed, at about \$0.028 per question. The whole final experiment cost about \$6.

It was also not possible on the available machine. The specified model needs about 8.5 GB of memory even in compressed form; the development machine has 8 GB shared with everything else, of which roughly 5.3 GB is usable.

The item was kept anyway, because its real value was never the cost saving. It is the claim that the system can run without depending on an outside provider, and nothing else in the iteration demonstrates that. So the connecting code was written and tested, and only the final run waits for suitable hardware. A test locks the specified model in place, because quietly substituting a smaller one that fits would mean reporting a different experiment under the same name.

5. The final experiment

The comparison everything else serves: five versions of the system, 34 questions, run three times each.

Why it runs only once. The 34 questions are the exam paper. Iteration 2 showed that the ranking of systems *changed* between a 10-question set and the 34-question set, so every look at the 34 costs some independence. The danger is not dishonesty — it is this: run, see the system lose, adjust the planner, run again, report the better number. Every individual step is defensible and the final figure is worthless.

Why the system is locked first. Locking is what makes “once” mean anything. Before the experiment, a document was committed recording every setting, and stating what would not be allowed afterwards. It also recorded, *in advance*, a known weakness: one required breakdown splits 14 questions into groups of 5 and 9, which is too few to detect anything but a huge effect. Writing that down beforehand means a flat result reads as “too small a sample”, and a large result does not get to count as confirmation either.

Why it was run three times rather than once. Not the same as running it repeatedly. Running a *locked* system three times and reporting the spread measures how much the numbers wobble; re-running after adjustments is what the “once” rule forbids. We had three separate reasons to need this, all discovered during the iteration — the marking, the paper-reading and the planning steps each turned out to be random to some degree.

6. The maintenance track

Four unresolved problems carried over from Iteration 2, each already narrowed down by a measurement:

1. **Add the missing graph links** — the first of the two repairs above.
2. **Fix code and data link extraction**, which was finding nothing.
3. **Fix one marking criterion** that disagreed with human marks.
4. **Have a second person mark some answers**, to check whether the marking is reliable at all.

Three of the four turned out to be refuted — the proposed repair did not work. That is reported as a result rather than smoothed over, and in two cases the failure was more informative than a success would have been.

The habits that shaped all of it

Four working rules emerged, three planned and one that appeared while doing the work.

Measure the instrument before measuring with it. A measurement taken with an uncalibrated instrument is worse than no measurement, because it looks like data.

Lock the system before any experiment that runs once. Protection by design rather than by resolving to be careful.

Keep independent work off the critical path. When local inference turned out to be impossible, nothing else stalled.

Run the cheap check before the expensive commitment. This was not planned and became the most valuable habit of the iteration. Six times, a free measurement taken *before* building something changed what got built — for example, tracing which section of each paper actually contains a code link, which revealed the problem was three separate faults rather than one. The single time this was skipped, a cost figure was estimated from memory at 13×; measured, it is 1.2×.

5.2 Terms

Basic vocabulary

1. **Retrieval** Searching the library for passages relevant to a question. The system can only cite what retrieval finds, so retrieval sets the ceiling on how good an answer can be.
2. **Knowledge graph** A network built by reading the papers, where the points are things (methods, problems, datasets, claims, limitations) and the connections are labelled relationships (“this method addresses that problem”, “this finding weakens that claim”). The project’s original bet was that searching this network would beat plain text search.
3. **Arm** One competing version of the system, run under identical conditions so the versions can be compared. Five arms were compared in the final experiment.

4. **Gold set** The 34 reference questions, each written with the answer’s required parts and the papers a correct answer must cite. It is the exam paper.
5. **Relational question** A question with two linked parts — “which methods, and what limits each”. The type the system did worst on, and the type Iteration 3 targets.
6. **Required-paper recall** The main score: of the papers a correct answer must cite, what fraction did the answer actually cite? A number between 0 and 1.
7. **p -value** The chance of seeing a result this strong if the system were really no better than the one it is compared against. Below 0.05 is the usual bar for calling a result real. Our headline result has $p = 0.012$.
8. **Agreement score** Used for marking. It compares two sets of marks on the same answers. 1.0 means perfect agreement; 0 means no better than guessing. This project treats a marking criterion as usable only above 0.6.

Components built in this iteration

1. **Agentic system** The system that plans, searches several times, checks itself, then answers. About 490 lines of code.
2. **Plan-scoring tool** Measures the plan instead of the answer (see Design, item 2). Split into a library of calculations plus a small script that reads files and prints a report, so the calculations can be tested on their own.
3. **Query-time write layer** Saves what the system works out while answering, marked as unreviewed and kept out of scoring (see Design, item 3).
4. **Arm-comparison tool** Compares versions *question by question* rather than by overall average, which is far more sensitive when there are only 34 questions. It produces the statistical test behind the headline result, and reports how much the numbers move between repeated runs so no figure appears without its uncertainty.
5. **Second-grader tooling** Picks a fair sample of answers, prints them *without showing any previous marks*, and compares the two sets of marks afterwards. Hiding the earlier marks is essential — shown a mark, a person tends to agree with it, which turns a measurement into a rubber stamp.
6. **Re-grading tool** Re-marks answers already saved to disk, without re-running the system. This is what makes a change to the marking scheme measurable at all: re-running everything would also produce different answers, so you could never tell whether a change in the marks came from the new scheme or from the system writing something different that time.

5.3 Fixes

Twenty-eight fixes. Two patterns are worth stating before the list.

Seven of them produced a believable wrong number rather than an error. No crash, no warning — just a figure that looked fine and was wrong. Those are the expensive ones, because nothing prompts anyone to look.

Four of the fixes that later caught a mistake looked like routine housekeeping when they were made — saving the marker’s written reasons, keeping the old marking scheme for comparison, checking whether the automatic checker worked, and building the plan-scoring tool

before the final experiment. None of them fixed anything visible at the time. Each is why a later error was caught rather than published.

Group 1 — Random settings were never turned off (7 fixes)

Nobody had ever set the “randomness” control on any AI call, so every call in the project used the provider’s default, which is random.

1. **The marking model was random**, so it gave different marks to the same answer on different days. Marking the same 34 answers three times with an identical scheme moved the agreement scores by up to 0.25 — and the whole distance to the “trustworthy” cutoff was only 0.26. The wobble was as big as the thing being measured.
2. **The paper-reading model had the same problem**, so the knowledge graph could never be rebuilt to match itself. Reading the same 21 papers twice gave 9 differences out of 147 facts. Fixed *before* rebuilding the whole library, so the expensive rebuild only had to happen once.
3. **The planner is random and turning the control off does not help**. Asked the same question three times, it produced three differently worded plans. Because those words become the search terms, different wording finds different papers — the same code scored 0.567 one run and 0.333 the next. Fixed by a design change instead: the system now always searches the original question first, which brought run-to-run variation from 0.050 to 0.000.
4. **Criteria were approved on a single lucky run**. Marking can now repeat, and a criterion is approved only if it passes on the *worst* run.
5. **The marker’s written reasons were thrown away**. The model was asked for a one-line reason with every mark and the code kept only the number. Those reasons are the only way we worked out why a rewritten marking scheme failed — they showed it penalising correct behaviour. A number cannot explain itself.
6. **There was no way to re-mark answers we already had**, so a change to the marking scheme could never be separated from the system writing different answers.
7. **Old marking schemes were not kept**, so there was nothing to compare against. This mattered: once randomness was fixed and all three versions were compared properly, the *original* scheme scored best, and the default was changed back to it.

Group 2 — The question sets were used wrongly (3 fixes)

There are two sets: 10 questions for development, 34 for the real test.

1. Scripts quietly used the 10-question file even when scoring a 34-question run, silently scoring a third of the data. They now pick the smallest file that covers every question, and stop with an error if none does.
2. Results depend on which set is used, and nobody was told. The two sets disagree about which marking criteria are trustworthy — one scores 0.79 on the 10 questions and 0.30 on the other 24. Both are now always printed together.
3. The main scoring script could only use the 10-question set. It is now selectable, and runs on a different set are labelled so they cannot be mistaken for development runs.

Group 3 — Finding code and data links (3 fixes)

This part records where a paper’s code and data can be downloaded. It was finding nothing at all — 0 out of 15 papers. There were three separate causes, not one.

1. **It searched the wrong sections.** Of the five papers that do state a code link, four state it somewhere it was not looking.
2. **It was asked for the information without being told what to collect.** The instructions listing which details to record were only sent when the section was also being searched for hardware details.
3. **The database could not store the most common answer.** Papers very often say data is “available from the authors on request”, and that was not an allowed value. Two of our reference answers say exactly that — which means **no possible result could ever have been marked correct**. The best achievable score was 4 out of 6 before the reading step was even involved.

Together these took code links from 0 to 2 and data availability from 0 to 2.

Group 4 — Contradiction-finding tools (4 fixes)

1. **Saved results had no record of which instructions produced them.** Testing new instructions would have returned nearly 17,000 *old* results and reported them as new — one command away from a completely false finding.
2. **The audit divided by the wrong total**, reporting “of 3,072 accepted” for a version that accepted 1,172.
3. **One hardcoded input and output file**, so auditing a second attempt would have destroyed the record of the first.
4. **Nobody had ever checked whether the checker worked.** Adding that check showed the automatic checker agreed 76.7% of the time — but only because 45 of 60 cases were “no contradiction”. On the 15 cases that genuinely *are* contradictions, it found 2. A checker like that reports a low score no matter what is true. Without this check we would have run it and reported the number.

Group 5 — A safety rule written down but never enforced (3 fixes)

The code promised that scoring only ever reads the reviewed data, never the data the system writes while answering. **Nothing actually checked.** The promise held only because nothing had ever written to that part — so the first time the system wrote anything, it could have improved its own marks.

1. One search component did not check, in two separate places.
2. The other search component had the same gap in both places.
3. Nothing stopped unreviewed data being promoted into the scored data. Promotion now refuses unless an approval flag is passed explicitly.

Verified by writing 38 real entries and confirming the scored total did not move (20,688 before and after) while the unfiltered total did (20,688 to 20,726). One honest limit: this only tested the first of the two places.

Group 6 — Plan-scoring measurements (2 fixes)

1. **One measurement was always zero and looked fine.** The reference data writes paper identifiers with a prefix; the system’s log writes them without. The two lists could never match, so the measurement returned 0.000 for every question — as a believable number, not an error.
2. **One measurement was meaningless.** “Does the plan cover the question’s parts?” scored a perfect 1.000. Testing it against *other questions*’ plans showed unrelated plans scoring 0.769 against 0.808 for the correct one — at the cutoff in use, every unrelated pair passed. Replaced with a comparison-based version, plus an automatic validity check that prints *before* the scores rather than after them.

Group 7 — Plumbing (5 fixes)

1. No way to re-read a few papers to test a change — only all 271. Testing a change now costs about \$0.36 instead of \$6.72.
2. The system had nowhere to record what it did, so the plan-scoring tool had nothing to read.
3. Two scripts needed the same report-writing code, which was marked private.
4. The model connection could not be pointed at a locally run model, and the randomness control could not be set at all.
5. Two scripts needed the same file-selection logic, but scripts cannot borrow code from each other, so it moved into the shared library.

Group 8 — Documentation that contradicted our own results (2 fixes)

1. **The project’s front page approved the wrong marking criteria** — it called one trustworthy that is not, and rejected one that is. It also reported 10 reference questions instead of 34, 23,460 graph entries instead of 27,777, 42 tests instead of 214, and a “next step” that had been finished two iterations earlier. Its opening stated the project’s central assumption as though it had not been tested.
2. **A section set up an experiment and never said it lost.** It explained the work done to make a fair comparison possible between the knowledge graph and plain citation links, and omitted that the comparison went against the graph.

One deliberate non-fix. The Iteration 2 report was *not* corrected, even though several of its numbers are now superseded. The Iteration 3 plan quotes those numbers to justify its own decisions, so editing them would retroactively change what the plan refers to and erase the record of what was known at the time. A report that silently updates itself cannot be checked. The corrections are recorded in the Iteration 3 report instead, as a table listing each superseded claim.

References

- [1] Edge et al., “*From Local to Global: A Graph RAG Approach to Query-Focused Summarization*”. Microsoft Research, 2024. [arXiv:2404.16130](https://arxiv.org/abs/2404.16130).